Splicing a base class subobject

Document #:	P3293R0
Date:	2024-05-19
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Peter Dimov <pdimov@gmail.com> Dan Katz <dkatz85@bloomberg.net> Barry Revzin <barry.revzin@gmail.com> Daveed Vandevoorde <daveed@edg.com>

Contents

§ 1 Introduction

There are many contexts in which it is useful to perform the same operation on each subobject of an object in sequence. These include serialization or formatting or hashing.

[P2996R3] seems like it gives us an ideal solution to this problem, in the form of being able to iterate over all the subobjects of an object and splicing accesses to them. However, it is not quite complete:

```
template <class T, class F>
void for_each_subobject(T const& obj, F f) {
    template for (constexpr auto sub : subobjects_of(^T)) {
         f(obj.[:sub:]); // this is valid syntax for non-static data
        members
                    // but is invalid for base classes subobjects
    }
}
```

Instead we have to handle bases separately from the non-static data members:

```
template <class T, class F>
void for_each_subobject(T const& obj, F f) {
    template for (constexpr auto base : bases_of(^T)) {
```

```
      f(static_cast<type_of(base) const&>(obj));
    }

    template for (constexpr auto sub : nonstatic_data_members_of(^T)) {
      f(obj.[:sub:]);
    }
}
```

Except this is now a normal `static_cast` and so requires access checking, thus prohibiting accessing private base classes.

We could avoid access checking by using a C-style cast:

```
template <class T, class F>
void for_each_subobject(T const& obj, F f) {
    template for (constexpr auto base : bases_of(^T)) {
      f((typename [: type_of(base) :]&)obj);
    }

    template for (constexpr auto sub : nonstatic_data_members_of(^T)) {
      f(obj.[:sub:]);
    }
}
```

But this opens up other problems: I forgot to write `const` and so now I accidentally cast away `const`-ness unintentionally. Oops. Not to mention that this cast actually works regardless of whether `base` refers to a base class of `T`, so it's not exactly the best programming practice.

On top of that, both the `static_cast` and C-style cast approaches suffer from having to correctly spell the destination type - which requires manually propagating the const-ness and value category of the object.

The way to avoid all of these problems is to just defer to a function template:

```
template <std::meta::info M, class T>
constexpr auto subobject_cast(T&& arg) -> auto&& {
    constexpr auto stripped = remove_cvref(^T);
    if constexpr (is_base(M)) {
        static_assert(is_base_of(type_of(M), stripped));
        return (typename [: copy_cvref(^T, type_of(M)) :])arg;
    } else {
        static_assert(parent_of(M) == stripped);
        return ((T&&)arg).[:M:];
    }
}

template <class T, class F>
void for_each_subobject(T const& obj, F f) {
    template for (constexpr auto sub : subobjects_of(^T)) {
      f(subobject_cast<sub>(obj));
```

```
    }
}
```

But this feels a bit silly? Why should we have to write this?

§ 2 Proposal

We propose to define `obj.[:mem:]` (where `mem` is a reflection of a base class of the type of `obj`) as being an access to that base class subobject, in the same way that `obj.[:nsdm:]` (where `nsdm` is a reflection of a non-static data member) is an access to that data member.

Additionally `&[:mem:]` where `mem` is a reflection of a base class `B` of type `T` should yield a `B T::*` with appropriate offset.

We argue that these are the obvious, useful, and only possible meanings of these syntaxes, so we should simply support them in the language.

The only reason this isn't initially part of [P2996R3] is that while there *is* a way to access a data member of an object directly (just `obj.mem`), there is *no* way to access a base class subobject directly outside of one of the casts described above. Part of the reason for this is that while a data member is always just an *`identifier`*, a base class subobject can have an arbitrary complex name.

This means that adding this support in reflection would mean that splicing can achieve something the language cannot do natively. But we don't really see that as a problem. Reflection is already allowing all sorts of things that the language cannot do natively. What's one more?

§ 3 References

[P2996R3] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, and Dan Katz. 2024-05-16. Reflection for C++26.
https://wg21.link/p2996r3